Table 1: The density objective against the methods it is built on. OGBench manipulation (proprioceptive input), success rate (%), 1M steps, reward-inference evaluation (z inferred from 10^4 relabelled samples). For each backbone: the published number, our reimplementation under one common protocol, and the same backbone trained with the density objective. Mean ± std over 3 seeds.

	FB			HILP			RLDP		
	published	ours	+ density	published	ours	+ density	published	ours	+ density
avg	17.5	15.2	—	35.2	30.7	30.9	12.5	12.7	18.6
cube-single	49.6 ±3.8	45.6 ±5.9	**90.9** ±1.4	74.2 ±3.5	71.1 ±5.7	77.0 ±3.5^{r}	19.8 ±2.4	27.6 ±3.7	53.3 ±3.2^{r}
cube-double	2.6 ±0.4	2.2 ±2.7	15.8 ±1.3	20.0 ±2.7	11.1 ±3.3	2.4 ±1.6	3.8 ±0.8	2.7 ±1.3	1.1 ±0.4
scene	12.8 ±1.6	10.2 ±4.1	—	43.8 ±1.9	39.3 ±5.7	42.0 ±0.9	11.6 ±1.6	10.2 ±3.2	12.2 ±4.7
puzzle-3x3	4.8 ±0.7	2.7 ±0.9	—	2.8 ±0.7	1.3 ±0.5	2.2 ±1.3	14.6 ±0.9	10.4 ±1.7	7.6 ±2.7

Published columns: TD-JEPA Table 1, 1M steps, 10 seeds, best cell of the hyperparameter grid, same reward-inference route; mean ± s.e. The five antmaze rows of that table are dropped: this paper is about manipulation, and the object variable that motivates the method does not exist in navigation. Ours and + density columns: our code, state observations to the forward network and actor, flow actor, last-3-checkpoint estimator (mean of the final three evaluation passes), 1M steps, 3 seeds. Baselines are fb_tdj, hilp_tdjc and rldp at its own default orthonormality coefficient 1.0 (exp/ortho1); TD-JEPA's grid used 0.01, which costs RLDP scene 10.2 → 4.7 and puzzle 10.4 → 2.4. Density columns: Density-FB = density_fb_z_state; Density-HILP = hilp_density_reg (cube-single) and hilp_density_tdjc (elsewhere); Density-RLDP = rldp_density_reg (cube-single) and rldp_density at ortho 1 (elsewhere). ^{r}: the task vector comes from a hindsight goal in the same trajectory, read out by the reward regression of Appendix **??**. Density cells without this mark keep the host method's own rule for choosing a task vector, which is what makes them level with the plain backbone — the matching ^{r} runs on cube-double, scene and puzzle-3x3 have not been done, so the two retrofit averages are not yet a measurement of the objective. Every column reads the full state: no method here is given object coordinates. Density-FB is density_fb_z_state, which exists only on the two cube environments — the scene and puzzle-3x3 runs at full state have not been done, so that column has no average. With object coordinates in the backward network the same method reaches 81.3±3.8 / 30.4±2.6 / 52.7±3.3 / 4.7±0.9 on the four tasks; that input costs it cube-single (90.9 → 81.3) and buys it cube-double (15.8 → 30.4). **avg** is the mean of the four rows, recomputed per column; no spread is carried through the recomputation. On cube-double, reward inference is capped near half of goal-conditioned success for every method: OGBench re-permutes the cube-to-target assignment at every reset, while the reward samples used to infer z encode one fixed assignment. Under goal conditioning Density-FB reaches 68.6±1.0 there and 93.1±1.5 on cube-single.